```
#!/usr/bin/env python3
"""
cross_auditor_orchestrator.py
------------------------------
NovusÆxenti / NovÆcopia Autonomous Orchestration & Auditing Stack
Component: Master Sovereign Orchestrator & Cross-Auditor Daemon

Unifies:
1. Tool-Call Interceptor & Telemetry Dashboard Stream
2. Divergence Detector (detects tool bypass, unauthorized deletion/move, skill refusal)
3. NanoClaw Routing Engine (hot-loads skills and tools per prompt)
4. ClearanceToken Handshake Engine for safe execution gatekeeping
"""

import os
import sys
import time
import json
import sqlite3
import argparse
from pathlib import Path
from datetime import datetime, timezone

SCRIPT_DIR = Path(__file__).parent.resolve()
if str(SCRIPT_DIR) not in sys.path:
    sys.path.insert(0, str(SCRIPT_DIR))

from tool_call_interceptor import ToolCallInterceptor, TelemetryEvent
from divergence_detector import DivergenceDetector
from nanoclaw_router import NanoClawRouter

DB_DIR = Path("/tmp/novae_sqlite")
LEDGER_DB = DB_DIR / "audit_ledger.db"

class ClearanceToken:
    def __init__(self, token_id: str, tool_name: str, status: str, risk_level: str):
        self.token_id = token_id
        self.tool_name = tool_name
        self.status = status
        self.risk_level = risk_level
        self.issued_at = datetime.now(timezone.utc).isoformat()

    def to_dict(self) -> dict:
```

```
        return {
            "token_id": self.token_id,
            "tool_name": self.tool_name,
            "status": self.status,
            "risk_level": self.risk_level,
            "issued_at": self.issued_at
        }

class CrossAuditorOrchestrator:
    def __init__(self, http_port: int = 8088, strict_mode: bool = False):
        self.interceptor = ToolCallInterceptor(http_port=http_port)
        self.detector = DivergenceDetector(strict_mode=strict_mode)
        self.router = NanoClawRouter()
        self._init_ledger_db()

    def _init_ledger_db(self):
        try:
            DB_DIR.mkdir(parents=True, exist_ok=True)
            conn = sqlite3.connect(str(LEDGER_DB))
            cur = conn.cursor()
            cur.execute("""
                CREATE TABLE IF NOT EXISTS audit_events (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    event_id TEXT UNIQUE,
                    timestamp TEXT,
                    caller TEXT,
                    tool_name TEXT,
                    status TEXT,
                    risk_level TEXT,
                    divergence_flag INTEGER,
                    parameters_json TEXT
                )
            """)
            conn.commit()
            conn.close()
        except Exception as e:
            sys.stderr.write(f"[Ledger DB Init Error] {e}\n")

    def _log_event_to_ledger(self, event: TelemetryEvent, has_divergence: bool):
        try:
            conn = sqlite3.connect(str(LEDGER_DB))
            cur = conn.cursor()
            cur.execute("""
```

```
            INSERT OR IGNORE INTO audit_events
            (event_id, timestamp, caller, tool_name, status, risk_level, divergence_flag,
parameters_json)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?)
        """, (
            event.event_id,
            event.timestamp,
            event.caller,
            event.tool_name,
            event.status,
            event.risk_level,
            1 if has_divergence else 0,
            json.dumps(event.parameters)
        ))
        conn.commit()
        conn.close()
    except Exception as e:
        sys.stderr.write(f"[Ledger DB Write Error] {e}\n")

  def evaluate_and_gate(self, user_intent: str, tool_name: str, parameters: dict, caller: str =
"claude_code_cli") -> tuple[ClearanceToken, str]:
    event = self.interceptor.intercept(tool_name, parameters, caller=caller)
    incident = self.detector.evaluate(user_intent, tool_name, parameters)

    if incident:
        event.status = f"FLAGGED_{incident.severity}"
        self._log_event_to_ledger(event, has_divergence=True)

        if incident.severity == "CRITICAL_HALT":
            token = ClearanceToken(
                token_id=f"TKN-REJECT-{event.event_id}",
                tool_name=tool_name,
                status="BLOCKED",
                risk_level="CRITICAL"
            )
            return token, f"EXECUTION HALTED: {incident.message}"
        else:
            token = ClearanceToken(
                token_id=f"TKN-WARN-{event.event_id}",
                tool_name=tool_name,
                status="APPROVED_WITH_WARNING",
                risk_level=event.risk_level
            )
```

```
            return token, f"WARNING: {incident.message}"

        event.status = "CLEARED"
        self._log_event_to_ledger(event, has_divergence=False)
        token = ClearanceToken(
            token_id=f"TKN-OK-{event.event_id}",
            tool_name=tool_name,
            status="APPROVED",
            risk_level=event.risk_level
        )
        return token, "Execution cleared by Cross-Auditor."

    def start_orchestrator(self):
        self.interceptor.start_dashboard_background()
        print(f"[✓] Cross-Auditor Orchestrator active. Dashboard on
http://127.0.0.1:{self.interceptor.http_port}")

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Cross-Auditor Orchestrator Master
Daemon")
    parser.add_argument("--port", type=int, default=8088, help="Dashboard port")
    parser.add_argument("--test", action="store_true", help="Execute complete orchestration test
pass")
    args = parser.parse_args()

    orchestrator = CrossAuditorOrchestrator(http_port=args.port)
    print("[*] Cross-Auditor Orchestrator loaded and operational.")
```